

Reflection 2 - Cisco Interface Config Parser

LIS875

Ivan Lesley Mercado

Project Rubric

Points	Professionalism	Revision	Complexity	Personal Success
21-25	Took all Check-ins seriously. Responses to peers were thoughtful and constructive.	Included a one-page reflection summarizing feedback and how it was incorporated.	Applied course concepts appropriately (variables, loops, logic, data parsing). Code runs without errors and the project remains focused.	The program successfully parses Cisco interface configurations, extracts key fields, performs basic validation checks, and generates example configuration or rollback commands.
11-20	Completed all Check-ins but responses to peers were minimal.	Included feedback summary but suggestions were only partially incorporated.	Some course concepts applied correctly but project functionality is incomplete.	Parser extracts some fields but validation or command generation features are incomplete.
0-10	Check-ins or peer responses missing.	Feedback summary missing.	Code does not run or project lacks clear focus.	Project does not successfully parse configuration data.

In the NOC enterprise environment where this project originates, the most common operational requests involve VLAN changes, MAC sticky resets, port security

additions or removals, and dot1x configurations. The six fields the parser extracts; interface name, description, mode, VLAN, port security, and admin state are the exact attributes an engineer needs to review before making any of these changes. They represent the minimum information required to understand a port's current state and to generate accurate change and rollback commands.

The conflict detection; flagging interfaces where both MAC sticky and 802.1X are configured on the same port, is considered a basic validation because it checks for a condition that should never exist in a correctly configured environment. It does not attempt to evaluate whether a VLAN assignment is correct for a given department or whether port security settings meet policy requirements. It simply catches a known misconfiguration at parse time so that downstream logic does not generate commands against a port in an ambiguous state.

Professionalism (self-score: 22/25): I completed all four project check-ins (Week 3 pitch, Week 6 revised pitch and rubric, Week 11 draft, and this final submission). For peer engagement, I was more active in some areas than others. On Week 3 I had difficulty finding time to review peers' proposals due to work commitments and being focused on my own pitch. By Week 6 I was able to comment on two peers' projects, one on marketing campaign data analysis and another on college enrollment sentiment analysis. I completed all submissions to the studio sessions but only attended 3 out of 4 of those sessions with my group on Zoom and was also active in commenting on those group discussions. Since Week 11, I have addressed all unread peer feedback from Weeks 3, 6, and 11, and incorporated their suggestions into the final version of the project.

Revision (self-score: 24/25): My professor's feedback shaped the project at every stage. On Week 3, the suggestion to use basic Python string and file processing instead of jumping to libraries like Pandas led me to build the parser around explicit string matching (startswith and exact == comparisons) rather than regex or heavy library abstractions. The advice to flip sanitization from "what to remove" to "what to keep" reinforced a whitelist-based approach to extracting only the fields that matter. On Week 6, the feedback to articulate why certain fields are "key" and validations are "basic" helped me ground the project in its real-world NOC context rather than treating it as a generic parsing exercise. On Week 11, the professor suggested adding manpage/help text, which was implemented as a HELP_TEXT constant accessible via --help, -h, or running without arguments. All professor feedback has been incorporated. Peer feedback was also addressed; a glossary was added to the README, inline code comments were expanded, and the rollback system was redesigned from full-restore to action-aware in response to a peer's suggestion to clearly document rollback behavior.

Complexity (self-score: 24/25): The project applies course concepts throughout. Variables and dictionaries store parsed interface data. A for loop with conditional logic (if/elif) walks each interface block once to extract fields. String methods (startswith, split, len slicing, exact comparison) handle all text matching. The

conflict detection logic uses conditional checks during parsing to flag interfaces where two mutually exclusive security methods are both present. List operations (append, extend) build command sequences. The enumerate function pairs interfaces with numbered indices for the selection menu. A set tracks already-selected interfaces to prevent duplicates. List comprehensions filter ports by security type for action-specific validation. A dictionary maps action numbers to the fields each action touches, driving the action-aware rollback logic. During testing, the program runs without errors and the project remains anchored on its stated goal.

Personal Success (self-score: 25/25): The program successfully parses Cisco interface configurations from a running-config text file, extracting all six key fields. It performs basic validation by detecting security conflicts at parse time, and enforces security-aware filtering, actions that require a specific security type (MAC sticky or dot1x) will automatically skip ports that don't match, preventing invalid commands from being generated. It generates CLI command sequences for VLAN changes, MAC sticky resets, port security conversions (MAC sticky to dot1x and vice versa), VLAN changes with automatic security conversion, and rollback commands based on the original parsed configuration. The program supports batch operations with dynamic port selection (single numbers, ranges, and mixed input like 1,3,6-10) and generates commands using Cisco's interface range syntax. Rollback is also made action-aware as it only restores the attributes that the action actually changed, and includes removal of newly applied configurations before restoring the originals. For example, if only a VLAN was changed, rollback only restores the original VLAN. If a security conversion was performed, rollback removes the new security method and restores the original. Ports with identical rollback configurations are grouped into shared interface range blocks to keep the output compact. MAC sticky resets do not generate rollback commands since the port ends up in the same state. The program also includes help text accessible via --help, -h, or running without arguments, and validates VLAN input to only accept integers.

Project Progression (updated parserv3 section):

parserv3 (current version): Introduced dynamic port selection supporting ranges and mixed input (e.g., 1,3,5,6-10,11-20). Command generation uses Cisco's interface range syntax so that a single command block can apply the same change to all selected ports at once. The workflow was restructured into a batch loop; select a set of ports, apply one action, review the generated commands and rollback, then optionally select another batch.

After the Week 11 draft submission, the following improvements were made:

Action-aware rollback: The original rollback restored the full configuration of every port regardless of what was changed. This was replaced with a targeted

approach where rollback only restores the attributes that the action actually modified. For security conversions (actions 3, 4, and 5), rollback first removes the newly applied security configuration before restoring the original. This was the most substantial code change post-Week 11 and required restructuring the `generate_rollback` function to accept the action number and use it to determine which fields to include and which removal commands to prepend.

Security-aware filtering: Actions 2 (MAC sticky reset) and 3 (MAC sticky to dot1x) now only run on ports that have MAC sticky configured. Action 4 (dot1x to MAC sticky) only runs on ports with dot1x. Action 5 (VLAN change with security conversion) skips ports with no security configured and directs the user to use VLAN change instead. Ports that don't meet an action's requirements are listed and skipped automatically, and if no valid ports remain, the action is cancelled. This prevents the program from generating commands that don't make sense for a port's current configuration.

Auto-flip for action 5: The original action 5 prompted the user to manually enter a target security type. This was replaced with automatic detection; the program reads each port's current security and converts to the opposite. MAC sticky ports convert to dot1x, dot1x ports convert to MAC sticky. Mixed batches are split into separate command blocks by security type.

VLAN input validation: Actions that require a VLAN number now validate that the input is an integer before proceeding, re-prompting if the input is invalid.

Help text: Running the program without arguments or with `--help` / `-h` displays a manpage-style help screen covering usage, selection syntax, and available actions.

The project is named `parserv3` to signify that despite all the QA testing and refinements, it remains a work in progress with room for further development.

How Feedback Was Implemented:

Professor Feedback:

Week 3: The professor advised flipping the approach from removing sensitive data to only copying known-safe information. This was implemented in the parser's design. Rather than processing every line of a config file and stripping out what shouldn't be there, the parser only extracts the six fields it explicitly looks for (name, description, mode, VLAN, security, admin state) and ignores everything else. Lines like passwords, SNMP strings, or IP addresses are never captured because they're never matched.

Week 3: The professor recommended using basic Python string and file processing before reaching for libraries like Pandas. This shaped the entire parsing approach. All field extraction uses `startswith` and `exact ==` string comparisons rather than regex or heavy library abstractions. `CiscoConfParse` is used only for its ability to identify parent-child relationships between interface lines and their indented children; the actual text matching is done with plain Python string methods.

Week 11: The professor also suggested showing help text when the program is run without arguments or with `--help` / `-h`. This was implemented by adding a `HELP_TEXT` constant and updating the main function to check for these flags, printing a description of the program, usage examples, selection syntax, and available actions.

Peer Feedback:

Week 3: A peer asked whether I had an end-result structure I was trying to match and what aspect of the workflow I was aiming to improve. The structured output (the interface dictionary with six fields) was designed from scratch based on what NOC engineers need when reviewing port configurations. The workflow improvement is eliminating manual review of large config blocks and automating command and rollback generation.

Week 6: The same peer asked whether common setups could be created as templates based on team or location. Production areas often share consistent VLAN settings, and template-based configuration could be a natural extension. For now, the program generates commands based on user input per batch because each request still requires individual review given real-world variability across floors and campaigns.

Week 11: Another peer recommended adding a glossary of networking terms and more explanations of terminology. A glossary was added to the README covering all key terms (VLAN, MAC sticky, dot1x, interface range, running-config, access/trunk mode, admin state, rollback, `conf t / end / wr`). Additional inline comments were added throughout the code explaining the reasoning behind key logic decisions such as conflict detection, the command skeleton pattern, and rollback grouping behavior.

Week 11: This same peer noted that full-restore rollback behavior should be clearly documented. This feedback was taken further than originally planned, rather than just documenting the behavior, the rollback was redesigned to be action-aware, only restoring what was actually changed. The original full-restore behavior is preserved only for action 6 (generate rollback only), where a complete restoration is the intended purpose.